

Proyectos

Pepe Calderón, Fernando Sola, Daniel Ayala, Ana Belén Sánchez, Inma Hernández, Margarita Cruz, Carlos Arévalo, David Ruiz



Escuela Técnica Superior de
Ingeniería Informática

Índice

1. Diagrama de clases	3
2. Intensión	4
3. Álgebra relacional	6
4. Script SQL para crear la base de datos	10
5. Script SQL para la carga inicial de datos	13
6. Consultas	16
7. Pruebas SQL	19

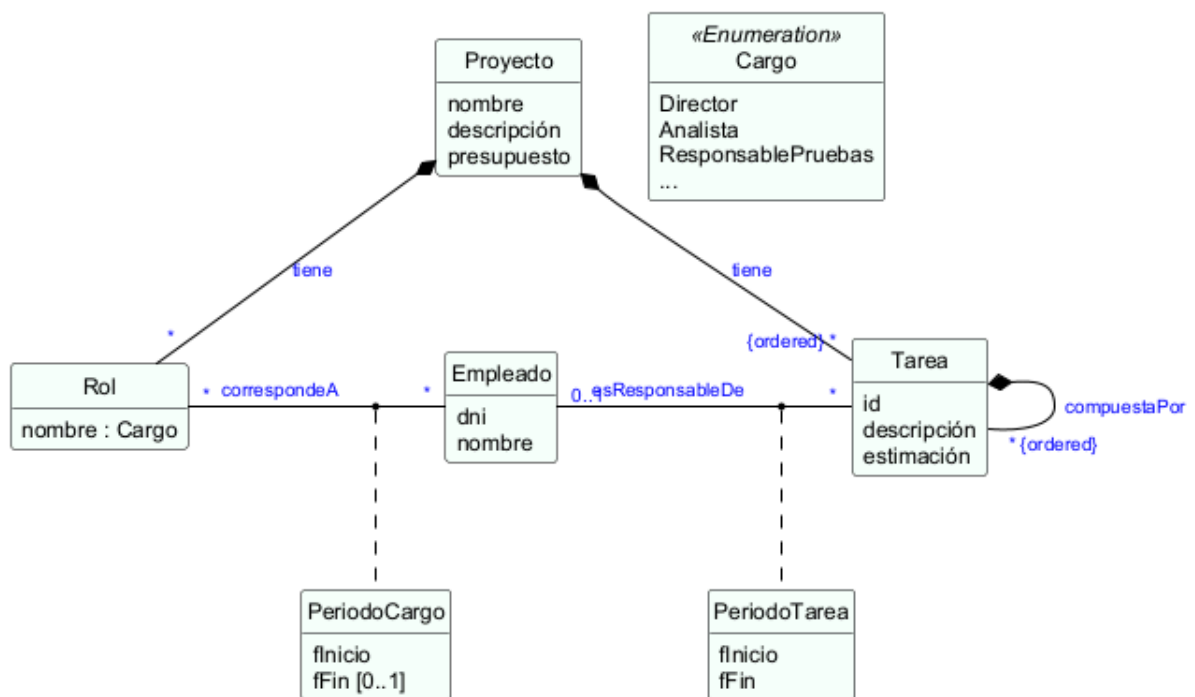
Requisitos La transcripción que aparece a continuación corresponde a una entrevista a una ingeniera de software que necesita un sistema de información para ayudarle en la gestión de sus proyectos.

- Pregunta: Bien, coménteme cuál sería el principal objetivo del sistema que usted necesita.
- Respuesta: Bueno, básicamente lo que quiero es un gestor de proyectos sencillo, que me permita gestionar las tareas asociadas y asignárselas a los empleados.
- P: Empecemos entonces por los proyectos, ¿qué información quiere que gestione el sistema sobre ellos?
- R: Básicamente, el nombre del proyecto, una descripción y el presupuesto que tiene. Puede que luego sea interesante añadirle más información, pero de momento me valdría con eso.
- P: De acuerdo, ha hablado antes de tareas. ¿Me puede explicar ese concepto?
- R: Sí, claro. Nosotros descomponemos los proyectos en tareas, lo que se conoce como WBS (Work Breakdown Structure). Cada tarea tiene un identificador que usamos para referirnos a ella, p.e. la tarea T-28F. También tienen una descripción y una estimación del coste de su realización. En el caso de que una tarea sea muy compleja, puede descomponerse en subtareas, que también pueden descomponerse recursivamente.
- P: Entiendo, ¿esas tareas tienen algún orden especial?
- R: El que le vayamos dando al crearlas, aunque luego podríamos cambiarlo.
- P: Entiendo, un proyecto tiene una secuencia de tareas que a su vez pueden tener una secuencia de subtareas y así recursivamente. Debemos conocer el orden. ¿Qué más hay que saber sobre las tareas?
- R: Pues que una vez que creamos la WBS, vamos asignando las tareas a los empleados.
- P: ¿En qué consiste una asignación de una tarea a un empleado?
- R: Básicamente, se encarga a un empleado la realización de una tarea con una fecha de inicio y otra de fin.
- P: ¿A cuántos empleados se le asigna una misma tarea?
- R: Cuando decidimos a quién asignársela, a uno sólo. Nuestras tareas simples están pensadas para que las pueda realizar un solo empleado.
- P: ¿Qué es una tarea simple?
- R: Una tarea que no está descompuesta en tareas más simples.
- P: Entiendo, sólo se asignan tareas simples, ¿no es así?
- R: Bueno, no necesariamente. Podemos asignar tareas complejas para su supervisión.
- P: Entiendo, se puede asignar cualquier tarea (compleja o simple) a cualquier empleado.
- R: Sí, aunque eso depende del rol que tenga el empleado en el proyecto.
- P: Explíqueme eso.
- R: Bueno, en cada proyecto se definen una serie de roles y cada empleado puede desempeñar distintos roles en distintos proyectos. Por ejemplo, uno puede ser el director de un proyecto y a la vez ser el responsable de pruebas de otro proyecto.
- P: Entiendo, ¿un mismo empleado puede jugar más de un rol en un mismo proyecto?
- R: No es frecuente, pero podría ocurrir, especialmente en proyectos pequeños.
- P: Una vez establecido un rol de un empleado en un proyecto, ¿puede cambiar?
- R: Sí, reasignamos los empleados según los proyectos que van saliendo.
- P: Entonces, le interesa saber desde qué fecha hasta qué fecha un empleado ha desempeñado un rol en un proyecto ¿no?

- R: Sí, quiero saber qué empleados han ido pasando por cada proyecto y en qué fechas.
- P: Entiendo que cada rol está asociado a un cargo predefinido, ¿es así?
- R: Sí, claro. Los nombres de los roles coinciden con los cargos habituales: director, analista, responsable de pruebas, etc. Tenemos definidos los roles según la norma ISO-9001.
- P: Muy bien, creo que con eso tengo para una primera versión. Gracias.

Modelo Conceptual

Diagrama de clases



Modelo Relacional

Intensión

```
Proyectos = { proyectoId, nombre, presupuesto }
  PK(proyectoId)
Roles = { rolId, proyectoId, nombre }
  PK(rolId)
  FK(proyectoId) / Proyectos
Tareas = { tareaId, proyectoId, orden, id, descripcion, estimacion }
  PK(tareaId)
  FK(proyectoId) / Proyectos
  AK(proyectoId, orden)
Subtareas = { subtareaId, tareaId, orden }
  PK(subtareaId, tareaId)
  FK(tareaId) / Tareas
  AK(subtareaId, orden)
Empleados = { empleadoId, dni, nombre }
  PK(empleadoId)
  AK(dni)
PeriodosCargos = { periodoCargoId, empleadoId, rolId, fInicio, fFin }
  PK(periodoCargoId)
  FK(empleadoId) / Empleados
  FK(rolId) / Roles
  AK(empleadoId, rolId)
PeriodosTareas = { periodoTareaId, empleadoId, tareaId, fInicio, fFin }
  PK(periodoTareaId)
  FK(empleadoId) / Empleados
  FK(tareaId) / Tareas
  AK(empleadoId, tareaId)

Proyectos = {
  (1, "Portal de alquileres", 50000.00),
  (2, "Gestor interno", 90000.00)
}

Roles = {
  (r1, 1, "Director"),
```

```
(r2, 1, "Analista"),
(r3, 2, "Responsable de pruebas")
}

Tareas = {
  (t1, 1, 1, "T-01", "Diseño del modelo", 40),
  (t2, 1, 2, "T-02", "Implementación de API", 80),
  (t3, 2, 1, "T-10", "Plan de pruebas", 60)
}

Subtareas = {
  (st1, t2, 1),
  (st2, t2, 2),
  (st3, t3, 1)
}

Empleados = {
  (e1, "11111111A", "Ana García"),
  (e2, "22222222B", "Luis Pérez"),
  (e3, "33333333C", "Marta López")
}

PeriodosCargos = {
  (pc1, e1, r1, "2024-01-01", NULL),
  (pc2, e2, r2, "2024-01-15", NULL),
  (pc3, e3, r3, "2024-02-01", NULL)
}

PeriodosTareas = {
  (pt1, e2, t1, "2024-02-01", "2024-02-15"),
  (pt2, e3, t2, "2024-02-16", NULL),
  (pt3, e1, t3, "2024-03-01", NULL)
}
```

Álgebra relacional

- Renombrado:

$$P \leftarrow \rho_{P(pid,pn,pres)} (Proyectos)$$

$$R \leftarrow \rho_{R(rid,pid,rn)} (Roles)$$

$$T \leftarrow \rho_{T(tid,pid,ord,cod,tdesc,est)} (Tareas)$$

$$E \leftarrow \rho_{E(eid,dni,en)} (Empleados)$$

$$PC \leftarrow \rho_{PC(pcid,eid,rid,fi,ff)} (PeriodosCargos)$$

$$PT \leftarrow \rho_{PT(ptid,eid,tid,fi,ff)} (PeriodosTareas)$$

$$ST \leftarrow \rho_{ST(stid,tid,sord)} (Subtareas)$$

- Empleados con roles en proyectos:

$$ER \leftarrow E \bowtie PC \bowtie R$$

ER = { eid, dni, en, pcid, rid, fi, ff, pid, rn }

ER = {
(e1, "11111111A", "Ana García", pc1, r1, "2024-01-01", NULL, 1, "Director"),

3. Álgebra relacional

```
{
  (e2, "22222222B", "Luis Pérez", pc2, r2, "2024-01-15", NULL, 1, "Analista"),
  (e3, "33333333C", "Marta López", pc3, r3, "2024-02-01", NULL, 2, "Responsable de pruebas")
}
```

- Empleados con tareas en proyectos:

$$ET \leftarrow E \bowtie PT \bowtie T$$

```
ET = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est }
```

```
ET = {
  (e2, "22222222B", "Luis Pérez", pt1, t1, "2024-02-01", "2024-02-15", 1, 1, "T-01", "Diseño
del modelo", 40),
  (e3, "33333333C", "Marta López", pt2, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación
de API", 80),
  (e1, "11111111A", "Ana García", pt3, t3, "2024-03-01", NULL, 2, 1, "T-10", "Plan de
pruebas", 60)
}
```

- Empleados que trabajan en proyectos (roles o tareas):

$$EP \leftarrow \Pi_{eid, en, pid}(ER) \cup \Pi_{eid, en, pid}(ET)$$

```
EP = { eid, en, pid }
```

```
EP = {
  (e1, "Ana García", 1),
  (e1, "Ana García", 2),
  (e2, "Luis Pérez", 1),
  (e3, "Marta López", 1),
  (e3, "Marta López", 2)
}
```

- Empleados que trabajan en proyectos (roles y tareas):

$$EP \leftarrow \Pi_{eid, en, pid}(ER) \cap \Pi_{eid, en, pid}(ET)$$

```
EP = { eid, en, pid }
```

```
EP = {
  (e2, "Luis Pérez", 1)
}
```

- Tareas asignadas a empleados en el proyecto 1:

$$TareasEmpleadoP1 \leftarrow \sigma_{pid=1}(ET)$$

```
TareasEmpleadoP1 = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est }
```

```
TareasEmpleadoP1 = {
  (e2, "22222222B", "Luis Pérez", pt1, t1, "2024-02-01", "2024-02-15", 1, 1, "T-01", "Diseño
del modelo", 40),
  (e3, "33333333C", "Marta López", pt2, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación
```

```
de API", 80)
}
```

- Número de tareas por empleado (proyecto 1):

$$NumTareasEmpleadoP1 \leftarrow \gamma_{eid, en, \rho_{total}}^{(COUNT(*))} eid, en(TareasEmpleadoP1)$$

```
NumTareasEmpleadoP1 = { eid, en, total }

NumTareasEmpleadoP1 = {
  (e2, "Luis Pérez", 1),
  (e3, "Marta López", 1)
}
```

- Listado de roles por proyecto:

$$ProyectosRoles \leftarrow P \bowtie R$$

```
ProyectosRoles = { pid, pn, pres, rid, rn }

ProyectosRoles = {
  (1, "Portal de alquileres", 50000.00, r1, "Director"),
  (1, "Portal de alquileres", 50000.00, r2, "Analista"),
  (2, "Gestor interno", 90000.00, r3, "Responsable de pruebas")
}
```

- Listado de tareas por proyecto:

$$ProyectosTareas \leftarrow P \bowtie T$$

```
ProyectosTareas = { pid, pn, pres, tid, ord, cod, tdesc, est }

ProyectosTareas = {
  (1, "Portal de alquileres", 50000.00, t1, 1, "T-01", "Diseño del modelo", 40),
  (1, "Portal de alquileres", 50000.00, t2, 2, "T-02", "Implementación de API", 80),
  (2, "Gestor interno", 90000.00, t3, 1, "T-10", "Plan de pruebas", 60)
}
```

- Número de subtareas por tarea:

$$NumSubtareas \leftarrow \gamma_{tid, \rho_{total}}^{(COUNT(*))} tid(T \bowtie ST)$$

```
NumSubtareas = { tid, total }

NumSubtareas = {
  (t2, 2),
  (t3, 1)
}
```

- Empleados con subtareas:

$$EST \leftarrow ET \bowtie ST$$

```
EST = { eid, dni, en, ptid, tid, fi, ff, pid, ord, cod, tdesc, est, stid, sord }
```

```
EST = {  
  (e3, "33333333C", "Marta López", pt2, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación  
de API", 80, st1, 1),  
  (e3, "33333333C", "Marta López", pt2, t2, "2024-02-16", NULL, 1, 2, "T-02", "Implementación  
de API", 80, st2, 2),  
  (e1, "11111111A", "Ana García", pt3, t3, "2024-03-01", NULL, 2, 1, "T-10", "Plan de  
pruebas", 60, st3, 1)  
}
```

Modelo Tecnológico

Script SQL para crear la base de datos

proyectos/createDB.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Noviembre de 2024  
-- Descripción: script para crear el esquema de la BD  
--  
  
-- Crear la base de datos  
DROP DATABASE IF EXISTS ProyectosDB;  
CREATE DATABASE ProyectosDB;  
USE ProyectosDB;  
  
-- Crear la tabla proyectos  
CREATE OR REPLACE TABLE projects (  
    project_id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255) NOT NULL,  
    budget DECIMAL(15, 2) NOT NULL,  
    CONSTRAINT ck_projects_budget CHECK (budget > 0)  
);  
  
-- Crear la tabla roles  
CREATE OR REPLACE TABLE roles (  
    role_id INT AUTO_INCREMENT PRIMARY KEY,  
    project_id INT,
```

```

    name VARCHAR(255) NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(project_id)
);

-- Crear la tabla tareas
CREATE OR REPLACE TABLE tasks (
    task_id INT AUTO_INCREMENT PRIMARY KEY,
    project_id INT,
    position INT NOT NULL,
    task_code CHAR(12) NOT NULL,
    description TEXT NOT NULL,
    estimate INT NOT NULL,
    FOREIGN KEY (project_id) REFERENCES projects(project_id),
    CONSTRAINT uq_tasks_project_position UNIQUE (project_id, position),
    CONSTRAINT uq_tasks_project_code UNIQUE (project_id, task_code),
    CONSTRAINT ck_tasks_position CHECK (position > 0),
    CONSTRAINT ck_tasks_estimate CHECK (estimate > 0)
);

-- Crear la tabla subtareas
CREATE OR REPLACE TABLE subtasks (
    subtask_id INT,
    task_id INT NOT NULL,
    position INT NOT NULL,
    PRIMARY KEY (subtask_id),
    FOREIGN KEY (subtask_id) REFERENCES tasks(task_id),
    FOREIGN KEY (task_id) REFERENCES tasks(task_id),
    CONSTRAINT uq_subtasks_task_position UNIQUE (task_id, position),
    CONSTRAINT ck_subtasks_position CHECK (position > 0)
);

-- Crear la tabla empleados
CREATE OR REPLACE TABLE employees (
    employee_id INT AUTO_INCREMENT PRIMARY KEY,
    national_id VARCHAR(20) NOT NULL UNIQUE,
    name VARCHAR(255) NOT NULL
);

-- Crear la tabla role_periods
CREATE OR REPLACE TABLE role_periods (
    role_period_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    role_id INT,
    start_date DATE NOT NULL,
    end_date DATE,
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),
    FOREIGN KEY (role_id) REFERENCES roles(role_id),
    UNIQUE (employee_id, role_id),
    CONSTRAINT ck_role_periods_dates CHECK (end_date IS NULL OR end_date >=
start_date)

```

```
);  
  
-- Crear la tabla task_periods  
CREATE OR REPLACE TABLE task_periods (  
    task_period_id INT AUTO_INCREMENT PRIMARY KEY,  
    employee_id INT,  
    task_id INT,  
    start_date DATE NOT NULL,  
    end_date DATE,  
    FOREIGN KEY (employee_id) REFERENCES employees(employee_id),  
    FOREIGN KEY (task_id) REFERENCES tasks(task_id),  
    UNIQUE (employee_id, task_id),  
    CONSTRAINT ck_task_periods_dates CHECK (end_date IS NULL OR end_date >=  
start_date)  
);
```

Script SQL para la carga inicial de datos

proyectos/populateDB.sql

```
--  
-- Autor: David Ruiz  
-- Fecha: Septiembre 2026  
-- Descripción: Procedimiento para poblar la BD de proyectos  
--  
  
USE ProyectosDB;  
  
DELIMITER //  
CREATE OR REPLACE PROCEDURE p_populate()  
BEGIN  
    SET FOREIGN_KEY_CHECKS = 0;  
    DELETE FROM task_periods;  
    DELETE FROM role_periods;  
    DELETE FROM subtasks;  
    DELETE FROM tasks;  
    DELETE FROM roles;  
    DELETE FROM employees;  
    DELETE FROM projects;  
    SET FOREIGN_KEY_CHECKS = 1;  
  
    INSERT INTO projects (project_id, name, budget) VALUES  
        (1, 'Proyecto A', 100000)
```

```

;

-- Insertar datos de ejemplo en la tabla roles
INSERT INTO roles (role_id, project_id, name) VALUES
    (1, 1, 'Director'),
    (2, 1, 'Analista'),
    (3, 1, 'Responsable de pruebas')
;

-- Insertar datos de ejemplo en la tabla tareas
INSERT INTO tasks (task_id, project_id, position, task_code,
description, estimate) VALUES
    (1, 1, 1, 'T-DI', 'Diseño de interfaz', 30),
    (2, 1, 2, 'T-DB', 'Desarrollo de BackEnd', 50),
    (3, 1, 3, 'T-T', 'Testing', 225),
    (4, 1, 4, 'T-TPU', 'Pruebas unitarias', 150),
    (5, 1, 5, 'T-TPI', 'Pruebas de integración', 75),
    (6, 1, 6, 'T-D', 'Despliegue', 10)
;

-- Insertar datos de ejemplo en la tabla subtareas
INSERT INTO subtasks (subtask_id, task_id, position) VALUES
    (4, 3, 1),
    (5, 3, 2)
;

-- Insertar datos de ejemplo en la tabla empleados
INSERT INTO employees (employee_id, national_id, name) VALUES
    (1, '12345678A', 'Juan Pérez'),
    (2, '87654321B', 'María López'),
    (3, '11223344C', 'Carlos García'),
    (4, '44332211D', 'Ana Rueda')
;

-- Insertar datos de ejemplo en la tabla role_periods
INSERT INTO role_periods (role_period_id, employee_id, role_id,
start_date, end_date) VALUES
    (1, 1, 1, '2023-12-01', null),
    (2, 2, 2, '2024-01-01', null),
    (3, 3, 3, '2024-01-01', null),
    (4, 4, 3, '2024-01-01', null)
;

-- Insertar datos de ejemplo en la tabla role_periods
INSERT INTO task_periods (task_period_id, employee_id, task_id,
start_date, end_date) VALUES
    (1, 1, 1, '2024-01-01', '2024-01-30'),
    (2, 2, 2, '2024-02-01', '2024-03-15'),
    (3, 3, 3, '2024-03-01', '2024-04-30'),
    (4, 4, 4, '2024-05-01', '2024-06-15')

```

```
;  
  
END //  
DELIMITER ;  
  
CALL p_populate();
```

Consultas

proyectos/queries.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Consultas SQL correspondientes al álgebra relacional de
-- proyectos

USE ProyectosDB;

-- ER: empleados con roles en proyectos
CREATE OR REPLACE VIEW v_employee_roles AS
SELECT e.employee_id AS eid, e.national_id, e.name AS en,
       pc.role_period_id AS pcid, pc.role_id AS rid,
       pc.start_date AS fi, pc.end_date AS ff,
       r.project_id AS pid, r.name AS rn
FROM employees e
JOIN role_periods pc ON pc.employee_id = e.employee_id
JOIN roles r ON r.role_id = pc.role_id;

SELECT * FROM v_employee_roles;

-- ET: empleados con tareas en proyectos
CREATE OR REPLACE VIEW v_employee_tasks AS
SELECT e.employee_id AS eid, e.national_id, e.name AS en,
       pt.task_period_id AS ptid, pt.task_id AS tid,
       pt.start_date AS fi, pt.end_date AS ff,
```

```

        t.project_id AS pid, t.position AS ord, t.task_code AS cod,
        t.description AS tdesc, t.estimate AS est
FROM employees e
JOIN task_periods pt ON pt.employee_id = e.employee_id
JOIN tasks t ON t.task_id = pt.task_id;

SELECT * FROM v_employee_tasks;

-- EP: empleados que trabajan en proyectos, por rol o por tarea
SELECT eid, en, pid
FROM v_employee_roles
UNION
SELECT eid, en, pid
FROM v_employee_tasks;

-- EP: empleados que trabajan en proyectos, tanto por rol como por tarea
SELECT eid, en, pid
FROM v_employee_roles
INTERSECT
SELECT eid, en, pid
FROM v_employee_tasks;

-- TareasEmpleadoP1: tareas asignadas a empleados en el proyecto 1
SELECT *
FROM v_employee_tasks
WHERE pid = 1;

-- NumTareasEmpleadoP1: Número de tareas por empleado en el proyecto 1
SELECT eid, en, COUNT(*) AS total
FROM v_employee_tasks
WHERE pid = 1
GROUP BY eid, en;

-- ProyectosRoles: roles de cada proyecto
SELECT p.project_id AS pid, p.name AS pn, p.budget AS pres,
       r.role_id AS rid, r.name AS rn
FROM projects p
JOIN roles r ON r.project_id = p.project_id;

-- ProyectosTareas: tareas de cada proyecto
SELECT p.project_id AS pid, p.name AS pn, p.budget AS pres,
       t.task_id AS tid, t.position AS ord, t.task_code AS cod,
       t.description AS tdesc, t.estimate AS est
FROM projects p
JOIN tasks t ON t.project_id = p.project_id;

-- NumSubtareas: Número de subtareas por tarea
SELECT t.task_id AS tid, COUNT(*) AS total
FROM tasks t
JOIN subtasks st ON st.task_id = t.task_id

```

```
GROUP BY t.task_id;
```

```
-- EST: empleados con subtareas
```

```
SELECT et.*, st.subtask_id AS stid, st.position AS sord
```

```
FROM v_employee_tasks et
```

```
JOIN subtasks st ON st.task_id = et.tid;
```

Pruebas SQL

proyectos/tests.sql

```
--
-- Autor: David Ruiz
-- Fecha: Septiembre 2026
-- Descripción: Tests negativos para ProyectosDB
--
USE ProyectosDB;

CREATE OR REPLACE TABLE test_results (
    test_id VARCHAR(20) PRIMARY KEY,
    test_name VARCHAR(200) NOT NULL,
    test_message VARCHAR(500) NOT NULL,
    test_status ENUM('PASS', 'FAIL', 'ERROR') NOT NULL,
    execution_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

DELIMITER //
CREATE OR REPLACE PROCEDURE p_log_test(p_id VARCHAR(20), p_message
VARCHAR(500), p_status VARCHAR(10))
BEGIN
    INSERT INTO test_results VALUES (p_id, SUBSTRING_INDEX(p_message, ':',
1), p_message, p_status, DEFAULT);
END //

CREATE OR REPLACE PROCEDURE p_test_ck01_budget()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('CK01', 'CK01: El
```

```

presupuesto debe ser positivo', 'PASS');
    CALL p_populate();
    INSERT INTO projects (name, budget) VALUES ('Proyecto inválido', 0);
    CALL p_log_test('CK01', 'ERROR: Se permitió un presupuesto no
positivo', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq01_task_position()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ01', 'UQ01: El
orden de las tareas es único por proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO tasks (project_id, position, task_code, description,
estimate) VALUES (1, 1, 'T-X', 'Duplicada', 10);
    CALL p_log_test('UQ01', 'ERROR: Se permitió duplicar el orden de una
tarea', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq02_task_code()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ02', 'UQ02: El
identificador de tarea es único por proyecto', 'PASS');
    CALL p_populate();
    INSERT INTO tasks (project_id, position, task_code, description,
estimate) VALUES (1, 10, 'T-DI', 'Duplicada', 10);
    CALL p_log_test('UQ02', 'ERROR: Se permitió duplicar el identificador
de tarea', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_ck02_period()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('CK02', 'CK02: El
fin de un periodo no puede preceder a su inicio', 'PASS');
    CALL p_populate();
    INSERT INTO task_periods (employee_id, task_id, start_date, end_date)
VALUES (1, 2, '2026-02-01', '2026-01-01');
    CALL p_log_test('CK02', 'ERROR: Se permitió un periodo con fechas
inválidas', 'FAIL');
END //

CREATE OR REPLACE PROCEDURE p_test_uq03_national_id()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION CALL p_log_test('UQ03', 'UQ03: El
DNI de empleado debe ser único', 'PASS');
    CALL p_populate();
    INSERT INTO employees (national_id, name) VALUES ('12345678A',
'Empleado duplicado');
    CALL p_log_test('UQ03', 'ERROR: Se permitió duplicar el DNI', 'FAIL');
END //

```

```
CREATE OR REPLACE PROCEDURE p_run_tests()
BEGIN
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        CALL p_log_test('POPULATE', 'ERROR: La carga inicial falló',
        'ERROR');
        SELECT * FROM test_results ORDER BY test_id;
    END;
    DELETE FROM test_results;
    CALL p_populate();
    CALL p_test_ck01_budget();
    CALL p_test_uq01_task_position();
    CALL p_test_uq02_task_code();
    CALL p_test_ck02_period();
    CALL p_test_uq03_national_id();
    SELECT * FROM test_results ORDER BY test_id;
    SELECT test_status, COUNT(*) total FROM test_results GROUP BY
test_status;
END //
DELIMITER ;
CALL p_run_tests();
```

i Info

Versión PDF disponible